

Lecture 11: Fetch–Decode–Execute Cycle (Instruction Cycle)

Learning Objectives

By the end of this lecture, students will be able to:

- Define the Instruction Cycle.
 - Explain the Fetch–Decode–Execute Cycle.
 - Understand the role of the Program Counter (PC), Instruction Register (IR), MAR, and MDR.
 - Explain how the Control Unit coordinates instruction execution.
 - Understand how the ALU executes arithmetic and logical operations.
 - Follow the complete execution of a program step by step.
 - Relate the instruction cycle to Assembly Language.
-

1. Introduction

Teaching Script

First 30-Second SEO Phrase

Naturally mention:

"In today's Computer Organization and Assembly Language lecture, we'll learn the complete Fetch–Decode–Execute Cycle, understand how the CPU executes instructions using the Program Counter, Instruction Register, MAR, MDR, Control Unit, ALU, and System Bus, with easy examples and diagrams."

In the previous lectures, we studied the CPU, the Arithmetic Logic Unit (ALU), the Control Unit (CU), CPU Registers, and the System Bus.

But now an important question arises:

When you click an application, type on your keyboard, or run a C++ program, how does the CPU actually execute those instructions?

Does it simply execute them randomly?

The answer is no.

Every processor follows a sequence of steps known as the **Fetch–Decode–Execute Cycle**, also called the **Instruction Cycle**.

Every single instruction—whether it's an addition, subtraction, data movement, or branch—is executed by repeating this cycle.

In today's lecture, we'll understand this complete process step by step with diagrams and practical examples.

Let's begin.

2. What is the Instruction Cycle?

Definition

The Instruction Cycle is the complete process through which the CPU fetches an instruction from memory, decodes it, executes it, and prepares to execute the next instruction.

In simple words:

Every instruction goes through three main stages:

Fetch → Decode → Execute

This process repeats continuously while the computer is running.

3. Why is the Instruction Cycle Needed?

Imagine a chef following a recipe.

For every dish, the chef:

1. Reads the next step.
2. Understands the instruction.
3. Performs the action.
4. Moves to the next step.

Similarly, the CPU:

1. Fetches the next instruction.
 2. Decodes its meaning.
 3. Executes it.
 4. Repeats the process.
-

4. Overall Instruction Cycle

Program

|



Memory

|



Fetch Instruction

|



Decode Instruction

|



Execute Instruction

|



Store Result (if needed)

|



Next Instruction

This sequence continues until the program ends.

5. Components Involved

During the instruction cycle, the following components work together:

- Program Counter (PC)
 - Memory Address Register (MAR)
 - Memory Data Register (MDR)
 - Instruction Register (IR)
 - Control Unit (CU)
 - Arithmetic Logic Unit (ALU)
 - Registers
 - Memory
 - System Bus
-

6. Stage 1 – Fetch

What Happens?

The CPU retrieves the next instruction from memory.

Step-by-Step

Step 1: The **Program Counter (PC)** contains the address of the next instruction.

Example:

PC = 1000

Step 2: The address is copied to the **Memory Address Register (MAR)**.

MAR $\leftarrow$ PC

Step 3: The Control Unit sends a **READ** signal through the Control Bus.

Step 4: Memory sends the instruction to the **Memory Data Register (MDR)**.

MDR = LOAD R1, A

Step 5: The instruction is copied into the **Instruction Register (IR)**.

IR $\leftarrow$ MDR

Step 6: The Program Counter increments to the next instruction.

PC = 1001

Fetch Diagram

Program Counter

|



MAR

|



Memory

|



MDR

|



IR

7. Stage 2 – Decode

Now the instruction is inside the **Instruction Register (IR)**.

Suppose the instruction is:

ADD R1, R2

The **Control Unit** examines the instruction and answers questions such as:

- Which operation is required?
- Which registers are involved?
- Does memory need to be accessed?
- Should the ALU perform addition, subtraction, or another operation?

The Control Unit then generates the required control signals.

Decode Diagram

Instruction Register

|



Control Unit

|



Generates Control Signals

8. Stage 3 – Execute

The CPU now performs the required operation.

Suppose:

R1 = 15

R2 = 25

Instruction:

ADD R1, R2

Execution:

$15 + 25 = 40$

The ALU performs the addition.

The result is stored back into a register.

R1 = 40

Execute Diagram

Registers

|



ALU

|



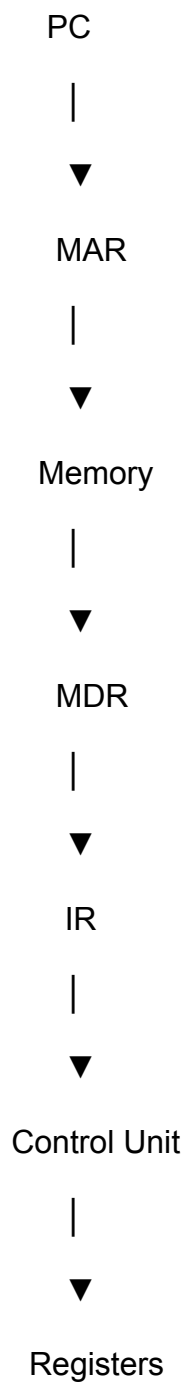
Result

|



Registers

9. Complete Instruction Cycle





10. Practical Example

Consider this C++ code:

```
int x = 10;
```

```
int y = 20;
```

```
int z = x + y;
```

A simplified execution flow is:

Fetch

Fetch the instruction from memory.

↓

Decode

Determine that it is an addition operation.

↓

Execute

Load the values into registers.

$R1 = 10$

$R2 = 20$

ALU computes:

$10 + 20 = 30$

The result is stored:

$R3 = 30$

Then it is written back to memory for **z**.

11. Example with Assembly Language

Consider the following MIPS instruction:

`add $t0, $t1, $t2`

Meaning:

$\$t0 = \$t1 + \$t2$

Execution steps:

1. Fetch the instruction from memory.
2. Decode the instruction.
3. Read `$t1` and `$t2`.
4. ALU performs the addition.
5. Store the result in `$t0`.
6. Continue to the next instruction.

This is the same Fetch–Decode–Execute Cycle.

12. Memory Access Example

Instruction:

LOAD R1, A

Execution:

PC

↓

MAR

↓

Memory



MDR



IR



Control Unit



Address of A



Memory



Data Bus



Register R1

Notice that some instructions require an additional memory access after decoding.

13. Branch Instruction Example

Suppose the instruction is:

BEQ R1, R2, LOOP

Meaning:

Branch to **LOOP** if **R1** equals **R2**.

Execution:

- Fetch instruction.
- Decode it.
- Compare **R1** and **R2**.
- If equal, load the address of **LOOP** into the Program Counter.
- Otherwise, continue with the next sequential instruction.

This demonstrates that the Program Counter does **not always increment**; branch and jump instructions can change its value.

14. Real-Life Analogy

Imagine a delivery company.

- **Order List** → Program in memory
- **Manager** → Control Unit
- **Delivery Address** → Program Counter
- **Delivery Vehicle** → Data Bus
- **Worker** → ALU
- **Storage Room** → Memory
- **Work Table** → Registers

Every order follows the same sequence:

1. Read the next order.
2. Understand the order.
3. Complete the task.
4. Move to the next order.

The CPU works in exactly the same way.

15. Why Is This Cycle So Important?

Without the Fetch–Decode–Execute Cycle:

- Programs could not run.
- The CPU would not know which instruction comes next.
- Memory and registers would not work together.
- The ALU would not know what operation to perform.

Every program—from a calculator to a modern operating system—depends on this cycle.



Lecture Summary

Today, we studied the Fetch–Decode–Execute Cycle, also known as the Instruction Cycle, which is the fundamental process every CPU follows to execute instructions.

We learned how the Program Counter (PC), Memory Address Register (MAR), Memory Data Register (MDR), Instruction Register (IR), Control Unit, ALU, Registers, Memory, and the System Bus all work together during instruction execution.

We also followed the execution of arithmetic, load, and branch instructions step by step and saw how the Program Counter controls the flow of a program.

In the next lecture, we'll study **Instruction Set Architecture (ISA)** and learn how processors define the instructions that programmers and compilers use to communicate with the hardware. We'll also compare **RISC** and **CISC** architectures, which are among the most important topics in Computer Organization.

Thank you for watching, and I'll see you in the next lecture.
